Kainodaros variklis

Kursinio darbo I dalis: projektavimo dokumentas

1. Problema ir idėja

Sistema vienu sakiniu: Sistema, skirta aptarnauti daugelį klientų ir automatiškai apskaičiuoti sąskaitų sumas smulkiajam verslui ar savarankiškai dirbantiems specialistams, remiantis jų kainoraščiais, nuolaidų taisyklėmis bei PVM tarifais, kartu pateikiant išsamią skaičiavimų išklotinę.

Problema ir dabartinis procesas: Daroma prielaida, kad šiuo metu smulkiojo verslo įmonė sąskaitų sumas skaičiuoja naudodamasi skaičiuokle arba rankiniu būdu. Dėl šio proceso kyla šios problemos:

- laikas apvalinamas nenuosekliai (kartais – iki artimiausios valandos, kartais – iki dienos);
- įvairios nuolaidos (už kiekį, lojalumo ir kt.) taikomos nenuosekliai; neaišku, ar jos sumuojamos, ar kokia tvarka taikomos;
- sąskaitoje nurodoma tik galutinė suma, todėl klientui sunku suprasti, kodėl jis moka būtent tokią sumą.

Nauda:

- Nuoseklumas. Tie patys pradiniai duomenys visada lemia tą patį rezultatą.
- Skaidrumas. Kiekviena nuolaida rodoma atskirai, nurodant jos taikymo priežastį, o netaikomos nuolaidos atveju pateikiamas paaiškinimas.
- Saugumas. Kiekvienas nuomininkas turi savo kainoraštį, nuolaidų parametrus bei PVM tarifą ir nemato kito nuomininko duomenų.
- Palaikomumas. Galima pridėti naują nuolaidų taisyklę nekeičiant jau esamų.

Naudotojai: Savininkas arba sąskaitas išrašantis asmuo. Jie pasirenka klientą, gauna kainos išklotinę ir išsaugo sąskaitą (klientai tiesiogiai su sistema nesąveikauja).

Prielaidos:

- Nuomininkas išrašo sąskaitas faktūras pagal valandinį įkainį. Visos sumos nurodomos eurais.
- Kiekvienas nuomininkas pats nustato savo kainoraštį, nuolaidų pakopas, didžiausią leistiną nuolaidą, kuponus ir PVM tarifą.
- PVM tarifas yra nuomininko konfigūracijos dalis. Demonstracijoje naudojami pavyzdiniai 21 % ir 0 % tarifai (pastarasis – PVM mokėtoju neįregistruotam nuomininkui).
- Šiam darbui atlikti pakanka dviejų nuomininkų.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Sąskaitos kainos skaičiavimas	Pateikti paslaugų eilutes su laiku. Įgauna kainos išskaidymą arba aiškų klaidos pranešimą	Pagrindinis modulis
Sąskaitos išsaugojimas ir peržiūra	Išsaugoti apskaičiuotą sąskaitą ir vėliau ją atidaryti	Pagalbinė funkcija
Nuomininko pradiniai duomenys	Naudoti nuomininko iš anksto paruošti kainoraštį, užsakovus, kuponus ir taisyklių parametru	Pagalbinė funkcija
Nuomininkų duomenų atskyrimas	Dirbant vieno nuomininko kontekste, negalima perskaityti ar pakeisti kito nuomininko duomenų	Pagalbinė funkcija

Į kursinio darbo apimtį neįeina: registracija ir prisijungimas, rolių sistema, kainoraščių ir taisyklių redagavimo sąsaja, PDF sąskaitų generavimas, apmokėjimų apskaita, e. sąskaitų formatai ir mokesčių ataskaitos, kelios valiutos, išbaigtas dizainas, diegimas debesyje.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Kainodaros modulis. Atsižvelgdamas į klientą, paslaugų pozicijas (įskaitant trukmę) ir pasirinktinį kuponą, jis apskaičiuoja sąskaitos sumą bei pateikia išsamų paaiškinimą, taikydamas konkretaus nuomininko taisykles. Jei kainos apskaičiuoti nepavyksta, sistema pateikia aiškų klaidos pranešimą ir nieko neišsaugo.

Logika, kurią reikės projektuoti ir testuoti: laiko apvalinimas, nuolaidos lygio pasirinkimas, nuolaidų suderinamumas, bendros nuolaidos limito taikymas, kuponų tikrinimas, sumų apvalinimas ir PVM apskaičiavimas.

Įvestis: tenant (X-Tenant-Id), customer identifier, invoice date, coupon code (optional), service lines (service code and duration in minutes)

```
{
  "clientId": "client-a-02",
  "invoiceDate": "2026-11-10",
  "promoCode": "WELCOME15",
  "lines": [ { "serviceCode": "DEV", "minutes": 1200 } ]
}
```

Išvestis: tarpinė suma, pritaikytos nuolaidos su sumomis, nepritaikytos nuolaidos su priežastimis, suma be PVM, PVM ir galutinė suma.

```
{
  "subtotal": 1000.00,
  "discounts": [
    { "rule": "VOLUME", "amount": 100.00 },
    { "rule": "PROMO:WELCOME15", "requestedAmount": 135.00, "amount": 100.00,
"note": "apribota bendros nuolaidos ribos" }
  ],
  "notApplied": [ { "rule": "LOYALTY", "reason": "nesuderinama su kuponu;
mažesnė nauda (45.00 < 135.00)" } ],
  "net": 800.00, "vat": 168.00, "total": 968.00
}
```

Veikimo eiga:

1. Nustatomas nuomininko kontekstas; patikrinami įvesties duomenys (laikas > 0, eilutės nėra tuščios).
2. Klientas, paslauga ir kuponas ieškomi tik to nuomininko duomenyse.
3. Kiekvienos eilutės laikas suapvalinamas (R1) ir apskaičiuojama tarpinė suma.
4. Pasirenkama nuolaida už kiekį (R2).
5. Patikrinamas kuponas (R5) ir nusprendžiama, kokia asmeninė nuolaida taikoma (R3).
6. Pritaikoma bendra nuolaidos riba (R4).
7. Suapvalinamos pinigų sumos ir apskaičiuojamas PVM (R6).
8. Grąžinamas išsamus rezultatas arba klaida; sąskaitą galima išsaugoti atskiru veiksmu.

Taisyklės arba sprendimo žingsniai

1. R1. Laiko apvalinimas. Kiekvienos eilutės laikas apvalinamas į viršų iki artimiausio 15 minučių intervalo (mažiausio apmokestinamo vieneto). Jei laikas ≤ 0, fiksuojama klaida.
2. R2. Nuolaida už kiekį. Iš nuomininko nuolaidų pakopų lentelės pasirenkama viena pakopa: aukščiausia pakopa, kurios riba yra ≤ bendras apmokestinamas laikas (riba įskaičiuojama; lygiai 10 valandų priskiriama pakopai „nuo 10 valandų“). Nuolaida skaičiuojama nuo tarpinės sumos.
3. R3. Asmeninių nuolaidų suderinamumas. Lojalumo nuolaida ir kuponas nėra sumuojami. Jei taikytinos abi, pasirenkama ta, kuri suteikia didesnę nuolaidos sumą (skaičiuojamą nuo sumos po

nuolaidos už kiekį pritaikymo); sumoms sutapus, taikomas kuponas. Kita nuolaida pažymima kaip „netaikyta“ nurodant priežastį.

4. R4. Bendras nuolaidų limitas. Visų nuolaidų suma negali viršyti nuomininko nustatyto limito (išreikšto procentine tarpinės sumos dalimi). Viršijus limitą, paskutinė taikyta nuolaida sumažinama taip, kad būtų tiksliai laikomasi limito.
5. R5. Kuponas. Kuponas turi priklausyti tam pačiam nuomininkui ir galioti sąskaitos faktūros išrašymo dieną. Priešingu atveju skaičiavimas nutraukiamas su klaidos pranešimu (vietoj to, kad būtų tęsiamas be nuolaidos).
6. R6. Kiekviena nuolaida ir PVM suma apvalinama iki 2 skaitmenų po kablelio naudojant Math.Round(suma, 2) su numatytąja MidpointRounding.ToEven taisykle (apvalinimas iki artimiausio lyginio). Tarpinės skaičiavimo reikšmės prieš apvalinimą išlaikomos pakankamu tikslumu. PVM skaičiuojamas vieną kartą nuo bendros sumos po nuolaidų pritaikymo, o ne atskirai kiekvienai eilutei.

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
S1. Įprastas atvejis	Studio A, `client-a-01`, be kupono, data 2026-11-10. Eilutės: DEV 490 min; CONS 120 min	Skaičiuoti kainą	DEV → 495 min = 8,25 val. × 50 = 412,50 €; CONS = 2 val. × 70 = 140,00 €. Bendras laikas – 10,25 val., todėl taikoma 5 % apimties nuolaida. Tarpinė suma – 552,50 €. Nuolaida: 552,50 × 5 % = 27,625 €, apvalinus iki 2 skaitmenų po kablelio pagal R6 taisyklę (iki artimiausio lyginio), gaunama 27,62 €. Suma be PVM – 524,88 €. PVM (21 %) – 110,22 €. Galutinė suma – **635,10** €.
S2. Ribinis atvejis arba konfliktas	Studio A, `client-a-02` (ištikimybės), kuponas WELCOME15, data 2026-11-10. Eilutė: DEV 1200 min	Skaičiuoti kainą	Tarpinė 1000,00; apimties nuolaida 10 % = 100,00 (likutis 900,00). Ištikimybė 45,00 ir kuponas 135,00 nesuderinami -> taikomas kuponas, ištikimybė „netaikyta“. Bendra nuolaida 235,00 viršija 20 % ribą (200,00) -> kupono nuolaida sumažinama iki 100,00. Be PVM 800,00; PVM 168,00; **iš viso 968,00**
S3. Klaida arba neįmanomas rezultatas	Studio A, `client-a-01`, kuponas SPRING10, sąskaitos data 2027-01-10 (kuponas galiojo iki 2026-12-31). Eilutė: DEV 60 min	Skaičiuoti kainą	**Klaida**, kaina neapskaičiuojama, sąskaita neišsaugoma
S4. Nuomininkų izoliacija	Studio A išsaugojo sąskaitą (ID = X). Užklausa su antraštė `X-Tenant-Id: studio-b`	Atidaryti sąskaitą X; bandyti skaičiuoti su paslauga DEV	Sąskaitos atidaryti negalima, paslauga DEV Studio B kainoraštyje nerasta, Studio A duomenys nepakito

Jei modulis naudoja AI: Netaikoma

4. Kokybės atributas

Pasirinktas atributas: palaikomumas – naują nuolaidos taisyklę galima pridėti nekeičiant esamų taisyklių.

Kodėl svarbus šiai sistemai: Jei kiekviena nauja taisyklė reikalautų keisti bendrąjį skaičiavimo kodą, kiltų rizika sugadinti esamus skaičiavimus.

Tikrinimo scenarijus ir sąlygos: Patikrinimui būtų pridėta 7 % nuolaida pirmajam užsakymui. Ji būtų skaičiuojama po kiekio nuolaidos ir priklausytų tai pačiai nesuderinamumo grupei kaip lojalumo nuolaida ir kuponas.

Sėkmės kriterijus:

- pakeitimas nekeičia jokių esamų taisyklių failų ir nekeičia klasės ‚PricingEngine‘
- visi anksčiau parašyti testai praeina be pakeitimų
- naujai taisyklei sukurti 3-5 testai (atitiktis, neatitiktis, konfliktas su kuponu, ...)

Numatytas projektavimo sprendimas: Kiekviena nuolaida yra atskira klasė, įgyvendinanti bendrą sąsają (‚IDiscountRule‘) ir turinti nustatytą prioritetą bei nesuderinamumo grupę. Pipeline randa taisykles sąraše ir konkrečioje grupėje parenka tą, kuri suteikia didžiausią naudą.

Kaip patikrinsiu vėlesniame etape: atliksiu aprašytą pakeitimą atskiroje šakoje (su ‚git diff --stat‘ ir viso testų rinkinio rezultatą).

Sprendimo kaina arba ribojimas: apima daugiau klasių ir netiesioginį vykdymą. Klaidos nustatant prioritetus ir grupavimą gali pakeisti rezultatą.

Antras atributas

Pasirinktas atributas: nuomininkų duomenų izoliacija (saugumas).

Kodėl svarbus šiai sistemai: sąskaitų ir užsakovų duomenys yra komerciškai jautrūs. Vieno nuomininko duomenų nutekėjimas kitam sugadintų pasitikėjimą sistema.

Tikrinimo scenarijus ir sąlygos: Du nuomininkai turi savo klientus ir paskyras. Iš Studio B bandoma nuskaityti, išvardyti ir modifikuoti Studio A įrašus (naudojant Studio A identifikatorius).

Sėkmės kriterijus: Kiekvienam viešosios paslaugų sluoksnio veiksmui, priimančiam įrašo identifikatorių arba grąžinančiam sąrašą, yra numatytas izoliacijos testas. Visi tokie veiksmai atmeta įrašą, priklausantį kitam subjektui.

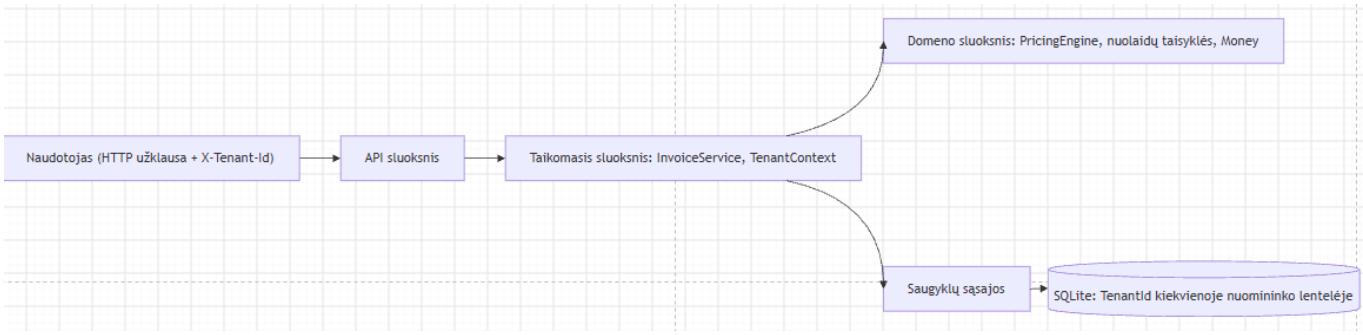
Numatytas projektavimo sprendimas: Kiekvienas nuomininko įrašas turi ‚TenantId‘. Serviso ir kiti sluoksniai kaip privalomą parametrą priima ‚TenantContext‘. (Gali būti pridėti papildomi apsaugos sluoksniai, pavyzdžiui, ‚TenantId‘ filtras ar pan.)

Kaip patikrinsiu vėlesniame etape: Parametrizuotų testų rinkinys, apimantis visus veiksmus, su atskirą testą, užtikrinantį, kad įrašas be nuomininko konteksto nebūtų išsaugotas.

Sprendimo kaina arba ribojimas: Kiekvienas naujas objektas privalo turėti ‚TenantId‘.

5. Pradinė sistemos struktūra

Paprasta schema



Sistemos dalis	Atsakomybė
API sluoksnis	Priima užklausas, pagal antraštę identifikuoja nuomininką, tikrina įvesties duomenis ir paverčia klaidas HTTP atsakymais.
Taikomasis sluoksnis	Įkelia nuomininko duomenis, iškviečia kainodaros mechanizmą ir išsaugo sąskaitą faktūrą. Saugo ‚TenantContext'.
Domeno sluoksnis	Verslo logika (R1 – R6). Veikia tik su perduotais duomenimis, nenaudoja duomenų bazės.
Infrastruktūra	SQLite saugyklos, pradinių duomenų užpildymas
Testai	Vienetiniai domeno testai be DB. Integraciniai testai su SQLite atmintyje (nuomininkų izoliacija)

Planuojamos technologijos ir pasirinkimo priežastys:

- C# / .NET: turiu šios aplinkos patirties, todėl daugiau laiko liks logikai ir testams.
- SQLite: nereikia atskiro DB serverio, todėl prototipą galima paleisti viena komanda.
- xUnit: standartinis .NET testų įrankis.
- Sąsaja: paprastas HTTP API. Grafinė sąsaja nereikalinga.

6. AI panaudojimas

AI rengiant šį dokumentą

[Nurodykite, ar naudojote AI. Jei taip, užpildykite lentelę.]

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
Claude (Anthropic): projekto idėjų sąrašas	AI padėjo pasirinkti projekto temą: jis pasiūlė 10-15 tinkamų variantų, iš kurių pasirinkau būtent šį.	Kitų temų (pvz., „Rezervavimas / paskola su konfliktų sprendimu“, „Tvarkaraščių / pamainų planuoklis“, „Studijų planuoklis“ ir t. t.) šiam projektui nepasirinkau, nes jos man pasirodė pernelyg nuobodžios arba per sudėtingos.	-
Claude (Anthropic): kainodaros variklio paaiškinimas	AI padėjo suprasti struktūrą ir tai, kaip veiks kainodaros variklis.	-	Patikrinau rankiniu būdu skaitydamas dirbtinio intelekto pateiktą turinį

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
Claude (Anthropic): šio dokumento juodraštis pagal šabloną	AI panaudotas kuriant šias dalis: „**Problema ir dabartinis procesas**“ (iš dalies), „**Įvestis**“ (JSON), „**Išvestis**“ (JSON), „**Veikimo eiga**“ (visiškai), „**Taisyklės arba sprendimo žingsniai**“ (iš dalies), „**Scenarijai būsimiems testams**“ (visiškai), „**Pradinė sistemos struktūra**“ (iš dalies), „**Būsimo prototipo veikimo scenarijus**“ (iš dalies).	Terminas „visiškai“ reiškia, kad turinį sugeneravo pats DI ir aš nieko papildomai nepridėjau. Terminas „iš dalies“ reiškia, kad DI pateikė atsakymą, tačiau jis man pasirodė nepakankamai išsamus arba netiko tokia forma, kokia buvo pateiktas, todėl jį performulavau ir (arba) papildžiau sava informacija.	Patikrinau rankiniu būdu skaitydamas dirbtinio intelekto pateiktą turinį ir lygindamas jį su reikalavimais bei savo projekto vizija.

Planuojamas AI naudojimas kuriant sistemą

Kur ir kam naudosiu AI: padėti rašyti pasikartojantį kodą (arba, jei pats nežinau, kaip įdiegti tam tikras sistemas, paprašyti AI tai paaiškinti), generuoti idėjas testavimo atvejams ir paaiškinti klaidas. Kainodaros taisyklių turinį bei tikėtinas testavimo reikšmes apibrėžiu pats, remdamasis šiuo dokumentu.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Prieš pritaikydamas bet kokį pakeitimą, jį peržiūrėsiu. Atlieku visus testus ir nenaudoju kodo, kurio negaliu paaiškinti.

Ar AI bus sistemos funkcionalumo dalis: ne

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
R1, R2, R6, `PricingEngine` pagrindas	S1 vienetinis testas praeina	1 etapas
Nesuderinamumo grupės (R3), riba (R4), kuponas (R5)	S2 ir S3 testai praeina, papildomi ribiniai testai	2 etapas
Saugojimas ir nuomininkai (`TenantId`, du nuomininkai, pradiniai duomenys)	S4 izoliacijos testai praeina	3 etapas
API: skaičiavimas, sąskaitos išsaugojimas ir peržiūra	Veikiantys užklausų pavyzdžiai	4 etapas
README, paleidimas viena komanda	Švari kopija repozitorijos paleidžiama pagal README	5 etapas

Būsimo prototipo veikimo scenarijus: Paleidžiu prototipą viena komanda ir naudodamasis Studio A išsiunčiu S2 užklausą, tikimasi, kad rezultatas bus apskaičiuotas teisingai. Tada išsiunčiu S3 užklausą (tikėdamasis gauti atsakymą PROMO_EXPIRED). Galiausiai, naudodamasis Studio B antrašte, bandau pasiekti Studio A paskyrą (tikėdamasis gauti atsakymą NOT_FOUND), kad pademonstruočiau, jog Studio B neturi to paties DEV kodo.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Kai taikomos kelios nuolaidų taisyklės ir apribojimai, rezultatai gali būti sunkiai nuspėjami.	Lentelinio testo (su ~10 derinių), apimančio rankiniu būdu apskaičiuotas vertes ir sąlygų patikras, kūrimas

Šaltiniai, jei naudojote

- https://dealhub.io/glossary/pricing-engine/
- https://www.vendavo.com/glossary/what-is-pricing-engine/
- Kursinio darbo I dalies užduoties aprašas ir šablonas